Ω-Uzu: 크로네커-슈어 곡률 트렐리스 양자화, 다이애딕 프리픽스 스트리밍 및 이중선형 상관 투기적 디코딩을 결합한 trymirai/uzu 차세대 추론 수학적 알고리즘 설계

[trymirai/uzu] 및 [trymirai/lalamo] 오픈소스 아키텍처 고도화 기술 명세서

2026년 10월 (참조 문헌: 2026년 7월 – 10월 최신 arXiv 논문)

요 약

본 문서는 Apple Silicon 통합 메모리(Unified Memory) 기반의 Rust/Metal 추론 엔진인 [trymirai/uzu] (v0.6.3)와 모델 최적화 프레임워크 [trymirai/lalamo]의 초당 토큰 생성 속도(TPS)를 극대화하고, 27B–300B+ 규모의 초대형 가중치(High-Weight) 모델을 단일 소비자용 디바이스에서 무손실에 가까운 정밀도로 고속 구동하기 위한 통합 수학적 알고리즘 **Ω-Uzu (Omega-Uzu)**를 제안한다. 현행 uzu는 4비트 비대칭 양자화(Mirai-M) 및 8비트 대칭 양자화(Mirai-L)와 블록 대각 랜덤 하다마드 변환(Block-Diagonal RHT), 그리고 블록 확산 초안기(DFlash)와 소형 자기회귀 어댑터(Weaver, [7])를 결합하여 높은 성능을 달성하고 있으나, (1) 투기적 디코딩 비활성화 시 메모리 대역폭 한계(Roofline Ceiling)에 수렴하는 기본 디코딩 속도, (2) 2–3비트 초저비트 영역에서의 룩업 테이블(LUT) 뱅크 충돌 및 런타임 역회전(Un-rotation) 오버헤드, (3) 755MB 규모의 외부 DFlash 초안 모델이 유발하는 +0.9 GB 상주 메모리 부담 및 고정 트리 예산(tree_budget=16)의 비효율성, (4) 장기 문맥(Long-Context)에서의 KV 캐시 대역폭 역전 현상이라는 네 가지 수학적·시스템적 병목을 안고 있다. 본 연구는 2026년 7월부터 10월 사이에 발표된 최신 arXiv 논문 방법론([7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24])을 수학적으로 결합하여 4대 핵심 알고리즘 계층을 도출하고, 오차 상한 증명과 Metal SIMD-group 커널 매핑 규격을 제시한다.

핵심 설계 요약 (Ω-Uzu 4대 수학적 기둥): (1) **KSC-CurveTQ + StagQ-AWSRC 가중치 압축**: [KronQ]/[CASA]의 크로네커 헤시안 조건수 보정과 [SchurQuant]의 슈어 보수 곡률을 [CurveTQ]의 비회전(Rotation-Free) 비터비 탐색 메트릭에 주입하고, [StagQ] 다이애딕 프리픽스 평면 및 [AWSRC] 의사난수 시드 잔차 복원(+0.162 bpw)을 결합하여 2.38 bpw에서 런타임 역회전 및 공유 메모리 LUT 충돌 없이 가중치 트래픽을 48.4% 절감한다.
(2) **DQ-ODP + ByteCross 동적 희소 교차 실행**: [Disaggregated Quantization (DQ)]으로 INT8 Neural Accelerator 프리필과 비양자화 활성화 디코딩을 분리하고, [ByteCross] 폐쇄형 교차 문맥 길이 $T^{\star}$에 따라 활성화 채널 희소성과 KV 희소성을 동적 결합한다.
(3) **LiLiCorr-StagQ 프리픽스 자가 투기 및 GDN Tree-Scan 검증**: [LiLiCorr] 단일 패스 이중선형 코사인 상관 행렬로 Weaver의 순차 지연을 제거하고, 외부 초안 모델 부재 시 [StagQ] 2비트 프리픽스 스트림과 [SparseSpec-L] 엔트로피 제어기를 통해 추가 메모리 0 MB로 자가 투기 디코딩을 수행하며, [GDN Tree-Scan]으로 결정론적 부동소수점 일관성을 보장한다.
(4) **AATC-OptR + ReadKV 질의 적응형 프리픽스 KV 캐시**: [AATC] 역방향 워터필링 및 [OptR] 출력 인지 회전 위에 [ReadKV] 질의 적응형 비트 평면 프리픽스 읽기를 적용하여 KV 메모리 읽기 대역폭을 4× 감축한다.

1 현행 trymirai/uzu 아키텍처의 수학적 병목 진단 (Baseline Analysis)

[trymirai/uzu] 엔진과 [trymirai/lalamo] 변환 툴킷은 Apple Silicon (M1–M5 및 A18/A19 Pro)의 통합 메모리 구조와 GPU Neural Accelerator에 맞춰 설계되어 있다 [4, 3]. 그러나 최근 실측 벤치마크([Mirai Metrics] 및 [Zenn M5 16GB 실측 분석])와 [GitHub 최근 PR (#893, #899, #900, #925)]을 수학적으로 분해하면 다음과 같은 4가지 구조적 한계가 드러난다.

1.1 단일 포워드 패스의 대역폭 천장(Roofline Wall)과 초대형 가중치 병목

단일 시퀀스 자기회귀 디코딩($B = 1$)에서 1개 토큰 생성 시 레이어 $l \in \{1, \dots, L\}$의 선형 투영 가중치 $W^{(l)} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$와 길이 T인 KV 캐시를 모두 DRAM에서 읽어야 한다. 유효 가중치 비트폭을 $\bar{b}_w$ (bits/weight), KV 캐시 평균 읽기 비트폭을 $\bar{b}_{\text{kv}}$, 전체 파라미터 수를 P, 모델의 레이어당 KV 차원을 d_{kv}라 할 때, 단일 포워드 패스당 전송 바이트 수 $\mathcal{M}_{\text{step}}(T)$는 다음과 같다:

$$\mathcal{M}_{\text{step}}(T) = \frac{P \cdot \bar{b}_w}{8} + \frac{2L \cdot T \cdot d_{\text{kv}} \cdot \bar{b}_{\text{kv}}}{8} + \mathcal{M}_{\text{meta}}. \tag{1}$$

Apple M5 기본형(16GB RAM, 대역폭 $\mathcal{B}_{\text{mem}} \approx 153$ GB/s)에서 Qwen3.5-9B의 Mirai-M 체크포인트(4비트 비대칭 양자화, 그룹 크기 $g = 64$, BF16 스케일 및 4비트 영점 포함 시 $\bar{b}_w \approx 4.61$ bpw, 가중치 용량 5.19 GB)를 투기적 디코

딩 없이($\tau = 1.0$ tokens/pass) 구동할 경우, 이론적 대역폭 상한은 $\mathrm{TPS}_{\max} = 153/5.19 \approx 29.48$ tok/s이며 실제 uzu 0.6.2/0.6.3 측정치는 26.9–27.0 tok/s (대역폭 효율 $\eta_{\mathrm{bw}} \approx 91.4\%$)로 MLX (26.0–26.6 tok/s)와 사실상 동일한 천장에 갇힌다 [6]. 즉, **가중치당 전송 비트 $\bar{b}_w$와 활성 읽기 비율 자체를 수학적으로 낮추지 않는 한**, 27B–70B+ 또는 300B MoE 모델은 단일 패스 지연시간과 메모리 상주 용량(예: Qwen3.6-27B Mirai-M의 15.70 GB 상주 메모리)으로 인해 일반 16GB–32GB 디바이스에서 구동이 불가능하거나 극심한 스와핑을 겪는다.

1.2 Mirai-M/L 블록 하다마드 변환 및 초기 트렐리스 커널(#899, #900)의 한계

lalamo의 현행 파이프라인은 워프 폭($w = 32$)에 맞춘 블록 대각 랜덤 하다마드 변환($\mathbf{H}_{32}$)으로 이상치(outlier)를 완화한 뒤, 입력 활성화 기반 2차 목적함수(YAQA)로 양자화하고 스케일/영점만 증류(QAD)한다 [4]. 또한 최근 uzu PR #899, #900, #925를 통해 도입 중인 트렐리스(Trellis) 양자화 커널은 2–3비트 압축을 목표로 하지만 다음 두 가지 수학적 손실이 발생한다:

1. **단방향 스칼라 민감도 왜곡 및 접미사(Suffix) 미보상**: 입력 활성화 공분산 $\mathbf{A} = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$만 반영하고 출력 그래디언트 공분산 $\mathbf{B} = \mathbb{E}[\mathbf{g}\mathbf{g}^\top]$ 및 미양자화 연속 접미사의 흡수 효과를 무시하여, 2비트 이하에서 대형 모델(예: 70B)의 Perplexity가 발산한다 [8, 12, 17].
2. **유클리드 비터비 탐색을 위한 런타임 회전 오버헤드**: 기존 트렐리스 코덱(QTIP, Proteus)은 블록 내부의 곡률 편차를 평탄화하기 위해 전체 또는 블록 직교 회전 $\mathbf{R}$을 곱하고 매 디코딩 스텝마다 이를 되돌리는 연산을 수행해야 한다 [23].

1.3 DFlash-Weaver 투기적 디코딩의 메모리 오버헤드, 결합성 및 비결정론적 드리프트

현행 uzu의 투기적 디코딩([3, 7])은 타겟 모델의 중간 레이어($l \in \{1, 5, 9, \ldots, 29\}$) 은닉 상태를 조건으로 받는 블록 확산 초안기 DFlash (755 MB 체크포인트, 실행 시 +0.9 GB 추가 메모리 점유)가 독립 주변 확률(marginals)을 출력하면, 56.7M 파라미터의 자기회귀 트랜스포머 Weaver ([7])가 상위 $K = 512$개 후보 위에서 제안 트리를 순차적으로 확장한다. 이 구조는 세 가지 한계를 지닌다:

1. **타겟 결합성 및 +0.9 GB 메모리 페널티**: 전용 DFlash 체크포인트가 없는 신규/파인튜닝 모델에서는 투기적 디코딩이 완전히 비활성화되어 TPS가 1/3–1/4로 급락하며, 16GB Mac 및 iOS(< 3 GB) 환경에서는 0.9 GB의 초안 모델 메모리가 치명적이다 [6].
2. **Weaver의 슬롯별 자기회귀 지연 및 고정 tree_budget**: Weaver가 트리 확장을 위해 슬롯마다 자기회귀 패스를 수행하며, shapes.json의 tree_budget이 하드웨어별 상수(예: M5–M6 모두 16)로 고정되어 있어, 엔트로피가 높은 자연어 산문(수락 길이 5.0–5.2)에서 불필요한 트리 검증 연산을 낭비한다 [9, 13].
3. **Greedy 디코딩 시 출력 불일치(Non-deterministic Drift)**: 동적 8비트 활성화 양자화와 병렬 트리 검증/Gated DeltaNet 상태 업데이트 과정에서 순차 FP32/BF16 경로와 부동소수점 누적 순서가 달라져, 온도 $T_{\mathrm{temp}} = 0$ (Greedy)에서도 200–300자 이후 출력이 갈라지는 현상이 발생한다 [6, 16, 18].

2 Pillar 1: 크로네커-슈어 곡률 트렐리스 및 다이애딕 시드 잔차 양자화 (KSC-CurveTQ + StagQ-AWSRC)

웨이트가 큰 모델(27B, 70B, 300B+ MoE)을 Apple Silicon에서 메모리 병목 없이 구동하려면 유효 비트폭을 4비트(Mirai-M)에서 2.2–2.4 bpw 영역(**Mirai-S 티어**)으로 낮추되, uzu가 지향하는 **(i) 공유 메모리 룩업 테이블(LUT) 뱅크 충돌 제로**, **(ii) 런타임 하다마드 역회전 오버헤드 제로**, **(iii) 2비트 붕괴(Perplexity 발산) 방지**를 동시에 달성해야 한다.

2.1 크로네커-팩터드 양방향 비간섭성 및 CASA 최적 비트 할당

레이어 l의 가중치 행렬 $W \in \mathbb{R}^{m \times n}$에 대해 양자화 오차 $\Delta W = \widehat{W} - W$가 전역 손실 $\mathcal{L}$에 미치는 2차 테일러 전개는 입력 활성화 $x \in \mathbb{R}^n$와 출력 그래디언트 $g = \nabla_y \mathcal{L} \in \mathbb{R}^m$의 크로네커 곱으로 근사된다 [8, 17]:

$$\mathbb{E}[\Delta\mathcal{L}] \approx \frac{1}{2}\,\mathrm{vec}(\Delta W)^\top (\mathbf{A} \otimes \mathbf{B})\,\mathrm{vec}(\Delta W) + \mathrm{tr}\left(\mathbf{G}^\top \Delta W\right), \quad \mathbf{A} = \mathbb{E}[\mathbf{x}\mathbf{x}^\top] \in \mathbb{R}^{n \times n}, \quad \mathbf{B} = \mathbb{E}[\mathbf{g}\mathbf{g}^\top] \in \mathbb{R}^{m \times m}, \quad (2)$$

여기서 $\mathbf{G} = \mathbb{E}[\mathbf{g}\mathbf{x}^\top]$은 [G^{2}PTQ] [20]가 도입한 블록 단위 갱신 1차 그래디언트 항이다.

표 1: 2026년 7월–10월 최신 arXiv 논문 기반 Ω-Uzu 통합 알고리즘 구성 매트릭스

계층 (Pillar)	참조 논문 (2026.07–10)	적용 대상 모듈	해결하는 uzu 병목 및 수학적 핵심 메커니즘
Pillar 1 초대형 가중치 초저비트 압축 (2.38 bpw)	[CurveTQ (2610.09212)] [23] [KronQ (2607.07964)] [8] [CASA (2609.25916)] [17] [SchurQuant (2608.15567)] [12] [G²PTQ (2609.31009)] [20] [StagQ (2610.05977)] [22] [AWSRC (2608.23144)] [14]	lalamo PTQ / uzu qmm_trellis 커널 (#899, #900, #925)	• 헤시안 $LDL^\top$ 대각 곡률 가중치를 비터비 브랜치 메트릭에 직접 주입하여 **회전 없는(Rotation-Free)** 2비트 트렐리스 탐색 달성 • 입출력 크로네커 헤시안($\mathbf{A}\otimes\mathbf{B}$) 조건수 $\sqrt{\kappa(\mathbf{A})\kappa(\mathbf{B})}$ 왜곡 보정 및 슈어 보수($\mathbf{H}_{G/S}$) 곡률로 2비트 붕괴 방지 • 2비트 기저 + 다이애딕 정밀화 평면(StagQ) 및 시드 생성 기저 잔차 복원(AWSRC, +0.162 bpw)으로 공유 메모리 LUT 충돌 제거
Pillar 2 위상 분리 및 동적 희소 교차	[DQ / ODP (2609.26333)] [18] [ByteCross (2609.33889)] [21] [KV-Wall (2609.30854)] [19]	uzu prefill & decode 스케줄러	• 프리필(INT8/FP4 연산 가속 + SSD 이중 버퍼 스트리밍 ODP)과 디코드(활성화 비양자화 + 2.38 bpw 가중치) 분리로 TTFT 1.78× 단축 • 문맥 길이 T와 바이트 교차점 $T^\star$에 따라 투영 가중치 활성화 희소성과 Split-K KV 희소성을 결합하여 14–26% 추가 가속
Pillar 3 자가·상관 투기적 디코딩	[LiLiCorr (2608.20530)] [13] [Weaver/TfM (2607.06763)] [7] [SpecQuant (2609.21704)] [15] [SparseSpec-L (2607.27735)] [9] [GDN Tree-Scan (2609.23900)] [16]	uzu speculator & gdn_verify	• DFlash 상위-K 주변 확률을 단일 패스 이중선형 코사인 유사도 행렬로 상관화하여 수락 길이 +7–19% 향상 (오버헤드 3%) • 전용 초안 모델 부재 또는 RAM 제약 시 StagQ 2비트 프리픽스 가중치 공유(SpecQuant) + 회수형 희소 KV(SparseSpec-L) 자가 투기 • GDN Tree-Scan 정적 인덱스 사전 계산으로 비트 단위 결정론 보장
Pillar 4 질의 적응형 프리픽스 KV	[AATC (2608.14191)] [11] [OptR (2608.02691)] [10] [ReadKV (2610.11245)] [24]	uzu kv_cache & attention	• W_O 투영 이후 어텐션 출력 왜곡을 최소화하는 직교 회전(OptR) 및 역방향 워터필링 변환 부호화(AATC, 5.8× 압축) • 8비트 점진적 코드로 저장된 KV 캐시에서 질의 벡터 q_t에 따라 평균 2–4비트 프리픽스만 선택적으로 읽어 지연시간 39% 절감

정리 (Theorem) 2.1 ([CASA]). 스칼라 민감도 왜곡 상한 및 폐쇄형 할당 [17]] 각 모듈의 헤시안을 단일 스칼라로 축약하는 기존 다중선택 배낭 문제(MCKP) 프록시는 전체 크로네커 이차 형식 대비 최대 $\sqrt{\kappa(\mathbf{A})\kappa(\mathbf{B})}$배의 곱셈적 왜곡(실제 LLM에서 $10^1 \sim 10^{13}$)을 유발한다. 모듈 l의 유효 크로네커 곡률 에너지를

$$\Omega_l \triangleq \left(\operatorname{tr}(\mathbf{A}_l)\operatorname{tr}(\mathbf{B}_l)\cdot\psi(\kappa(\mathbf{A}_l),\kappa(\mathbf{B}_l))\right)^{1/2}, \quad \psi(\kappa_A,\kappa_B)=\frac{(\kappa_A+1)(\kappa_B+1)}{4\sqrt{\kappa_A\kappa_B}} \tag{3}$$

로 정의하면, 목표 평균 비트 예산 $\bar{b}_{\text{target}}$ 하에서의 연속 완화 최적 비트폭 $b_l^\star$는 다음의 폐쇄형 해(Closed-form Solution)를 갖는다:

$$b_l^\star = \bar{b}_{\text{target}} + \frac{1}{2\ln 2}\ln\left(\frac{\Omega_l/P_l}{\prod_{k=1}^{L}(\Omega_k/P_k)^{P_k/P}}\right), \quad P=\sum_{k=1}^{L}P_k. \tag{4}$$

lalamo 변환기는 식 (4)의 1단계 해를 구한 뒤, 2단계로 엔드투엔드 KL 발산을 최소화하는 이산 국소 탐색을 수행하여 각 모듈을 $\{2,3,4\}$ bit로 배정한다. 이때 [arXiv:2607.08734]의 통계적 상전이 분석에 따라 어텐션의 W_Q, W_K 투영은 최소 3비트 이상으로 보호하고, 전체 파라미터의 약 68%를 차지하는 FFN 투영($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$) 및 MoE 전문가 가중치를 2비트 기저로 압축한다.

2.2 슈어 보수(Schur-Complement) 곡률 가중 비회전 트렐리스 탐색 (KSC-CurveTQ)

최근 uzu에 추가된 simdgroup trellis gemm (#900) 및 재시작 블록 패킹(#925)을 극대화하기 위해, [CurveTQ] [23]와 [SchurQuant] [12]를 결합한다. 입력 차원 n을 크기 $B_{\text{col}}=64$인 블록 G와 아직 양자화되지 않은 연속 접미사

(Suffix) S로 분할하자:

$$\widetilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A}_{GG} & \mathbf{A}_{GS} \\ \mathbf{A}_{SG} & \mathbf{A}_{SS} \end{bmatrix}, \quad \widetilde{\mathbf{A}} \triangleq \mathbf{A} + \lambda_{\text{reg}}\mathbf{I}. \tag{5}$$

기존 오차 피드백(GPTQ/YAQA)은 블록 G 내부의 이산 결정을 내릴 때 연속 접미사 S가 흡수할 수 있는 최적 연속 반응 $\Delta W^{\star}_{:,S} = -\Delta W_{:,G}\mathbf{A}_{GS}\mathbf{A}_{SS}^{-1}$을 무시하거나 블록 내부 곡률 변동을 버린다. [SchurOpt] [12]에 의해 접미사 S의 최적 연속 보상을 대수적으로 소거하면, 블록 G의 정확한 유효 이차 곡률 행렬은 **슈어 보수(Schur Complement)** $\mathbf{H}_{G/S}$로 주어진다:

$$\mathbf{H}_{G/S} = \mathbf{A}_{GG} - \mathbf{A}_{GS}\mathbf{A}_{SS}^{-1}\mathbf{A}_{SG} \in \mathbb{R}^{64\times 64}. \tag{6}$$

이제 $\mathbf{H}_{G/S}$에 대해 하삼각-대각 분해 $\mathbf{H}_{G/S} = \mathbf{L}_G\mathbf{D}_G\mathbf{L}_G^{\top}$ (단, $\mathbf{L}_G$는 주대각선이 1인 하삼각 행렬, $\mathbf{D}_G = \text{diag}(d_1, \ldots, d_{64})$)를 수행한다. [CurveTQ] [23]가 증명한 핵심 정리에 따르면, 오차 피드백 하에서 블록 G의 양자화 손실은 각 좌표 $j \in \{1, \ldots, 64\}$의 반올림 오차에 곡률 가중치 $d_j > 0$를 곱한 가중 제곱합으로 정확히 분해된다.

명제 (Proposition) 2.2 (비회전 곡률 가중 비터비 탐색과 랜덤 회전의 등가성 [23, 12, 20])**.** 출력 행 i의 크기 64인 블록 가중치 벡터 $w_{i,G} \in \mathbb{R}^{64}$를 상태 공간 $\mathcal{S}$ ($|\mathcal{S}| = 2^{14}$)를 갖는 트렐리스 부호기로 양자화할 때, [G^{2}PTQ]의 신뢰영역 스케일 $\eta_i \in (0, 1]$이 적용된 1차 그래디언트 보정 목표점을

$$\tilde{w}_{i,j} = w_{i,j} - \eta_i \frac{(\mathbf{B}_{ii})^{-1}G_{i,j}}{d_j} - \sum_{k<j}(\mathbf{L}_G)_{j,k}(w_{i,k} - \hat{w}_{i,k}) \tag{7}$$

로 정의하자. 이때 비터비(Viterbi) 동적 계획법의 브랜치 메트릭 $\mathcal{M}_{\text{branch}}(s_{j-1} \to s_j)$를 유클리드 거리 대신 **크로네커-슈어 곡률 가중 메트릭**

$$\mathcal{M}_{\text{branch}}\left(s_{j-1} \xrightarrow{c_j} s_j\right) = \mathbf{B}_{ii} \cdot d_j \cdot \left(\tilde{w}_{i,j} - \alpha_i\beta_j\mathcal{Q}_{\text{quant}}(\text{idx}(s_{j-1}, c_j))\right)^2 \tag{8}$$

로 정의하고, 블록당 14비트 초기 상태 $s_0^{\star} = \arg\min_{s_0\in\mathcal{S}} J_{\text{Viterbi}}(s_0)$를 저장(0.22 bpw)하면, 런타임 직교 회전(RHT) 없이도 네이티브 기저(Native Basis)에서 회전 기반 트렐리스 양자화(QTIP, Proteus)와 동일한 블록 내 곡률 적응성을 달성하며, 2비트 정확도를 $+1.0$–3.0%p 향상시킨다.

여기서 α_i, β_j는 인수분해된 행·열 스케일 필드이며, $\mathcal{Q}_{\text{quant}}(\cdot)$는 캘리브레이션 분포로부터 결정론적으로 구성된 128-엔트리 경험적 분위수(Quantile) 테이블이다 [23]. uzu의 Pack trellis restart blocks back to back (#925) 구조에 14비트 초기 상태 $s_0^{\star}$를 헤더로 패킹하면, Metal 디코더 커널(#900)은 **어떠한 런타임 하다마드 역회전 연산도 수행하지 않고** 레지스터 시프트와 128-엔트리 상수 메모리 분위수 참조만으로 최고 속도의 GEMV/GEMM을 수행한다.

2.3 StagQ 다이애딕 프리픽스 스트림 및 AWSRC 시드 잔차 복원 (LUT 충돌 제거)

uzu 팀이 기술 블로그([4])에서 지적한 바와 같이, 일반 벡터 양자화(VQ)는 GPU 공유 메모리(Threadgroup Memory) 룩업 테이블 접근 시 뱅크 충돌(Bank Conflict)을 일으키며, 디바이스 메모리 상황에 따라 2비트/3비트/4비트 체크포인트를 별도로 다운로드하는 것은 비효율적이다. 이를 해결하기 위해 [StagQ] [22]의 다이애딕 프리픽스 구조와 [AWSRC] [14]의 코드북 없는 시드 잔차 코덱을 결합한다:

$$\widehat{W}_{\Omega}^{(b)} = \underbrace{\widehat{W}_{\text{base}}^{(2\text{-bit})}}_{\text{KSC-CurveTQ / StagQ Base}} + \underbrace{\sum_{r=1}^{b-2} 2^{-r}{}_r \odot \mathbf{P}^{(r)}}_{\text{StagQ Dyadic 1-Bit Refinement Planes}} + \underbrace{\sum_{m=1}^{M_{\text{seed}}} \gamma_{g,m}\,(\sigma_{g,m})}_{\text{AWSRC Seeded Residual Sidecar (0.162 bpw)}}, \tag{9}$$

여기서 $\mathbf{P}^{(r)} \in \{-1, +1\}^{m\times n}$는 다이애딕(Dyadic) 스텝 스케줄을 따르는 1비트 정밀화 비트 평면이므로, 단일 체크포인트 스트림의 앞부분 2비트만 읽으면 2비트 모델로 동작하고 추가 1비트 평면을 읽으면 즉시 3비트/4비트 모델로 복원된다(룩업 테이블 불필요). 나아가 2비트 기저가 남기는 잔차 $R = W - \widehat{W}_{\text{base}}^{(2\text{-bit})}$에 대해, [AWSRC] [14]는 명시적 코드북을 메모리에 저장하는 대신 결정론적 의사난수 시드 선택자 $\sigma_{g,m} \in \{0, \ldots, 2^s - 1\}$와 저비트 스케일 $\gamma_{g,m}$만 사이드카(≈ 0.162 bpw, BF16 대비 0.8% 용량)로 저장한다. Metal 커널 내부에서는 스레드 레지스터 내 비트 해시 연산(Philox/SplitMix32 변형)으로 정규직교 기저 $(\sigma_{g,m})$를 즉석 생성(On-the-fly)하므로 **공유 메모리 뱅크 충돌이 0건**이면서 BF16 대비 PPL 격차의 88.2%, KL 발산 격차의 78.9%를 복구한다 [14].

3 Pillar 2: 위상 분리 양자화(DQ-ODP) 및 동적 희소 교차(ByteCross) 실행 엔진

3.1 프리필-디코드 위상 분리(Disaggregated Quantization)와 비결정론적 드리프트 해결

[Disaggregated Quantization (DQ)] [18]은 프리필($T \gg 1$, 연산 병목 $\mathcal{O}(TH^2)$)과 디코드($T = 1$, 대역폭 병목 $\mathcal{O}(H^2 + TH)$)의 최적 수 체계가 본질적으로 다름을 증명한다. 이를 uzu의 Apple M5 아키텍처에 매핑하면 다음 두

가지 혁신이 이루어진다:

1. **디코드 단계 활성화 비양자화(Format Disaggregation)를 통한 완벽한 결정론 회복**: uzu는 프리필 및 대규모 트리 검증($T \geq 16$)에서는 M5 Neural Accelerator의 INT8 GEMM 경로(블록 크기 32 RHT + 동적 8비트 활성화 양자화)를 유지하여 최고 연산 처리량을 얻되, 단일/소규모 디코딩 단계에서는 활성화 양자화를 해제하고 FP16/BF16 레지스터 누적과 2.38 bpw 가중치 디코딩을 직접 결합한다. [18]에 따르면 디코드 단계에서만 활성화 양자화를 제거해도 추론 비용 증가 없이 디코드 중심 태스크 정확도가 크게 향상되며, 특히 [Zenn 실측] [6]에서 보고된 **Greedy 디코딩 시의 출력 분기(200–300자 이후 불일치)를 근본적으로 제거**한다.
2. **오프로드 분리 프리필(ODP) 이중 버퍼 스트리밍**: 27B–70B 모델을 16GB–32GB Mac에서 구동할 때, 상주 메모리에는 2.38 bpw 디코드 체크포인트(예: 27B 기준 8.11 GB)만 유지하고, 프리필 전용 고속 가중치(INT8/FP4)는 Apple 내장 NVMe SSD(5–7 GB/s)에서 2개 레이어 크기의 고정 링 버퍼로 비동기 DMA 스트리밍하여 연산과 입출력을 중첩한다(Qwen3.8-27B 8K 문맥 기준 TTFT $1.78\times$ 가속 [18]).

3.2 ByteCross 기반 활성화 채널 희소성과 KV 희소성의 폐쇄형 교차 스케줄링

디코딩 중 매 스텝의 메모리 트래픽은 가중치 투영 항과 KV 캐시 항으로 구성된다. [ByteCross (arXiv:2609.33889)] [21]와 [KV-Wall (arXiv:2609.30854)] [19]을 결합하여, 활성화 채널 유지 비율을 $\rho_{\text{act}} \in (0,1]$, 어텐션 점수 기반 KV 선택 유지 비율을 $\rho_{\text{kv}} \in (0,1]$이라 하자. 레이어당 투영 가중치 바이트를 B_{proj}, 토큰당 KV 바이트를 $b_{\text{kv}}^{\text{tok}}$라 할 때, 두 희소화 기법이 절감하는 바이트 수가 정확히 같아지는 **폐쇄형 바이트 교차 문맥 길이(Byte Crossover Context Length)** $T^\star$는 다음과 같다 [21]:

$$\Delta B_{\text{proj}}(\rho_{\text{act}}) = \Delta B_{\text{kv}}(T^\star, \rho_{\text{kv}}) \implies (1-\rho_{\text{act}}) B_{\text{proj}} = (1-\rho_{\text{kv}})\, b_{\text{kv}}^{\text{tok}}\, T^\star \implies T^\star = \frac{1-\rho_{\text{act}}}{1-\rho_{\text{kv}}} \cdot \frac{B_{\text{proj}}}{b_{\text{kv}}^{\text{tok}}}. \tag{10}$$

동일한 Split-K Metal 어텐션 커널 위에서 두 분기를 합성할 때의 유효 가속비 상한 $\mathcal{S}_{\text{comp}}(T)$는 고정 커널 런치 오버헤드 $c_{\text{act}}, c_{\text{kv}}$를 포함하여 다음과 같이 유도된다:

$$\mathcal{S}_{\text{comp}}(T) = \frac{B_{\text{proj}} + b_{\text{kv}}^{\text{tok}} T}{\rho_{\text{act}} B_{\text{proj}} + \rho_{\text{kv}} b_{\text{kv}}^{\text{tok}} T + \mathcal{B}_{\text{mem}}\left(c_{\text{act}} \mathbb{I}[\rho_{\text{act}} < 1] + c_{\text{kv}} \mathbb{I}[T > T_{\text{min}}]\right)}. \tag{11}$$

uzu 런타임은 단기 문맥($T < T_{\text{min}}$)에서는 투영 가중치 타일 스킵($\rho_{\text{act}} \approx 0.72$, SwiGLU 게이트 활성화 크기 상위 채널 선택)만 활성화하여 고정 오버헤드 c_{kv}를 제거하고, 중·장기 문맥($T \geq T_{\text{min}}$)에서는 4개의 어텐션 싱크(Sink) 토큰 + 최근 윈도우 + 어텐션 점수 상위 KV 블록을 선택하는 Split-K 커널을 동시 가동하여 단일 분기 대비 14–26% 더 빠른 디코딩 속도를 달성한다 [21].

4 Pillar 3: 이중선형 주변확률 상관(LiLiCorr) 및 프리픽스 자가 투기 디코딩

4.1 Weaver를 대체하는 LiLiCorr 단일 패스 이중선형 우도 상관 트리 구축

현행 uzu의 DFlash-Weaver ([7, 3])에서 블록 확산 초안기 DFlash는 미래 위치 $t \in \{1, \ldots, \gamma\}$에 대해 독립 주변 로짓 $z_t \in \mathbb{R}^{|\mathcal{V}|}$을 병렬 생성하고, 각 위치의 상위-$K$ 후보 토큰 집합 $\mathcal{C}_t = \{v_{t,1}, \ldots, v_{t,K}\}$를 추출한다. 기존 Weaver (56.7M 파라미터)는 조건부 의존성을 복원하기 위해 슬롯마다 자기회귀적으로 트리를 확장하므로 초안 단계에 순차 지연이 발생한다.

이를 [LiLiCorr (arXiv:2608.20530)] [13]의 단일 패스 이중선형 우도 상관(Lightweight Likelihood Correlation) 모델로 치환한다. 단일 경량 네트워크 패스가 모든 슬롯 $t \in \{1, \ldots, \gamma\}$와 후보 $k \in \{1, \ldots, K\}$를 동시에 처리하여 각 후보마다 두 개의 저차원 방향 벡터 $\mathbf{u}_{t,k}^{\text{in}}, \mathbf{u}_{t,k}^{\text{out}} \in \mathbb{R}^{d_c}$ ($d_c \ll d_{\text{model}}$, 예: $d_c = 64$)를 방출한다. 인접한 두 슬롯 t와 $t+1$의 후보 쌍 $(v_{t,k}, v_{t+1,k'})$ 간 전이 상관 점수 행렬 $\mathbf{S}_t \in \mathbb{R}^{K \times K}$는 단일 배치 행렬 곱(Batched GEMM)으로 한 번에 계산된다:

$$\mathbf{S}_t(k,k') = \tau_{\text{corr}} \frac{\left\langle \mathbf{u}_{t,k}^{\text{out}}, \mathbf{u}_{t+1,k'}^{\text{in}} \right\rangle}{\left\| \mathbf{u}_{t,k}^{\text{out}} \right\|_2 \left\| \mathbf{u}_{t+1,k'}^{\text{in}} \right\|_2}, \quad \forall t \in \{1, \ldots, \gamma - 1\}. \tag{12}$$

따라서 길이 $m \leq \gamma$인 임의의 후보 경로 $\mathbf{y}_{1:m} = (v_{1,k_1}, \ldots, v_{m,k_m})$의 결합 로그 우도 대리 점수(Surrogate Joint Log-Likelihood) $\mathcal{J}(\mathbf{y}_{1:m})$는 지수적 결합 분포를 직접 실체화하지 않고도 다음과 같이 가산적으로 분해된다:

$$\mathcal{J}(\mathbf{y}_{1:m}) = \sum_{t=1}^{m} \ln p_{\text{DFlash}}(v_{t,k_t}) + \sum_{t=1}^{m-1} \ln \frac{\exp\left(\mathbf{S}_t(k_t, k_{t+1})\right)}{\sum_{j=1}^{K} \exp\left(\mathbf{S}_t(k_t, j)\right)}. \tag{13}$$

$\mathbf{S}_t$가 단일 패스에서 병렬로 모두 계산되므로, 트리 구성은 GPU 상의 초경량 동적 계획법/탐욕적 상위-B_{tree} 서브트리 추출로 귀결되어 블록당 지연시간의 3%만 소모하면서 평균 수락 토큰 길이(Acceptance Length)를 +7–19% 향상시킨다 [13].

4.2 StagQ-SpecQuant 무오버헤드 프리픽스 자가 초안 및 SparseSpec-L 동적 호라이즌 제어

외부 DFlash 초안 모델(755 MB)이 제공되지 않는 커스텀 모델이거나 16GB Mac / iPhone과 같이 $+0.9$ GB의 추가 메모리가 부담스러운 환경을 위해, [SpecQuant] [15], [StagQ] [22], [SparseSpec-L] [9]을 결합한 **Zero-Extra-Memory 자가 투기적 디코딩(Self-Speculative Decoding)**을 설계한다:

1. **다이애딕 프리픽스 가중치 공유 초안(StagQ + SpecQuant)**: 타겟 모델이 StagQ 3비트 또는 4비트 스트림($\widehat{W}_{\Omega}^{(4)}$)으로 메모리에 적재되어 있을 때, 별도의 초안 가중치를 메모리에 올리지 않고 **동일한 메모리 버퍼의 상위 2비트 프리픽스**($\widehat{W}_{\mathbf{base}}^{(\text{2-bit})}$)만 읽고 [SparseSpec-L]의 회수형 희소 KV 캐시(상위 중요 토큰 비율 $\rho_{\text{draft_kv}} \approx 0.15$)를 결합하여 초안 토큰을 생성한다. 초안 모델과 타겟 모델이 동일한 부모 가중치 매니폴드를 공유하므로 토크나이저 불일치가 없고 높은 수락률을 보장하며 추가 메모리 점유율이 0 MB이다 [15, 22].
2. **SparseSpec-L 온라인 엔트로피 기반 동적 트리 예산 제어**: uzu의 정적 shapes.json (tree_budget: 16)을 대체하기 위해, [SparseSpec-L] [9]의 스텝별 효율 정리를 적용한다. 초안 1스텝의 상대 비용을 $c_{\text{draft}} = T_{\text{draft}}/T_{\text{verify}}$라 할 때, 위치 t의 정규화 주변 엔트로피 $H_t = -\frac{1}{\ln K}\sum_{k=1}^{K} \tilde{p}_{t,k} \ln \tilde{p}_{t,k}$로부터 한계 수락 확률 $\hat{\alpha}_t = \exp(-\beta_H H_t)$를 추정한다. 스텝별 기대 속도 향상을 최대화하는 최적 투기 깊이 $\gamma^\star$와 동적 트리 예산 B_{tree}는 다음과 같이 결정된다 [9]:

$$\gamma^\star = \max\left\{ \gamma \in \{1, \dots, \gamma_{\max}\} \;\middle|\; \prod_{t=1}^{\gamma} \hat{\alpha}_t > \frac{c_{\text{draft}}}{1 + \gamma c_{\text{draft}}} \right\}, \tag{14}$$

$$B_{\text{tree}}(H_{1:\gamma^\star}) = \text{clip}\left(\left\lfloor B_0 \sum_{t=1}^{\gamma^\star} \hat{\alpha}_t (1 + \lambda_w H_t) \right\rfloor, B_{\min}, B_{\max} \right).$$

이를 통해 엔트로피가 낮은 코드 생성에서는 깊은 체인($\gamma^\star \uparrow$, 좁은 폭)을, 엔트로피가 높은 한국어·일본어 산문에서는 조기 중단 및 적응형 분기 폭을 자동 선택하여 낭비되는 검증 연산을 제거한다.

4.3 GDN Tree-Scan: 하이브리드 Gated DeltaNet의 비트 단위 결정론적 트리 검증

Qwen3.6-27B와 같은 순환-어텐션 하이브리드 모델에서 트리 검증 시 비결정론적 출력 변화가 발생하는 문제를 해결하기 위해, [GDN Tree-Scan (arXiv:2609.23900)] [16]의 경로 병렬 상태 복원 규격을 uzu의 Metal 커널에 도입한다. 트리 내 후보 노드 i의 부모 노드를 $\pi(i)$, 루트에서 i까지의 조상 경로를 $\mathcal{P}(i) = (r = u_0, u_1, \dots, u_d = i)$라 할 때, Gated DeltaNet의 상태 전이 식은 감쇠 게이트 $\alpha_{u_s} \in (0,1)$와 하우스홀더 유사 변환 행렬 $\mathbf{T}_{u_s} = \alpha_{u_s}(\mathbf{I} - \beta_{u_s}\mathbf{k}_{u_s}\mathbf{k}_{u_s}^\top)$에 대해 다음과 같이 표현된다:

$$\mathbf{S}_i = \left(\prod_{s=1}^{d} \mathbf{T}_{u_s} \right) \mathbf{S}_{\text{root}} + \sum_{s=1}^{d} \left(\prod_{j=s+1}^{d} \mathbf{T}_{u_j} \right) \beta_{u_s} \mathbf{v}_{u_s} \mathbf{k}_{u_s}^\top. \tag{15}$$

[GDN Tree-Scan] [16]에 따라 (1) 트리 위상으로부터 정적 Gather 인덱스를 사전 계산하여 브랜치별 합성곱(Short Convolution) 히스토리 윈도우의 탭 연산 순서와 캐스팅 경계(Cast Boundary)를 순차 디코딩과 비트 단위로 동일하게 고정하고, (2) 수락된 최종 체인 $\mathcal{P}(i^\star)$에 대해서만 디바이스 내부에서 상태를 커밋(Accepted-Chain-Only State Publication)하며, (3) uzu의 Gumbel 커플링 샘플러([5])에서 상위 24비트만 유지하고 $u \in [2^{-24}, 1-2^{-24}]$로 클램핑하는 규칙과 결합함으로써, **투기적 디코딩 유무와 무관하게 100% 동일한 출력 재현성**을 보장한다.

5 Pillar 4: 출력 인지 변환 부호화(AATC-OptR) 및 질의 적응형 프리픽스 KV 캐시(ReadKV)

장기 문맥 추론에서 KV 캐시 대역폭을 줄이기 위해 [AATC (arXiv:2608.14191)] [11], [OptR (arXiv:2608.02691)] [10], [ReadKV (arXiv:2610.11245)] [24]를 하나의 파이프라인으로 통합한다.

5.1 OptR 어텐션 동치 키 재매개변수화 및 AATC 역방향 워터필링 (5.8배 압축)

어텐션 헤드 h의 키 행렬 $\mathbf{K}_h \in \mathbb{R}^{T \times d_h}$는 채널별 거대 상수 오프셋 $\boldsymbol{\mu}_K = \frac{1}{T}\sum_{t=1}^{T} \mathbf{k}_{h,t}$를 포함한다. [OptR] [10]은 소프트맥스의 이동 불변성($\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$)을 활용하여, 어텐션 분포를 전혀 바꾸지 않고 키의 동적 범위를 축소하는 재매개변수화 및 헤드별 직교 회전 $\mathbf{U}_h, \mathbf{V}_h \in \mathcal{O}(d_h)$를 적용한다:

$$\widetilde{\mathbf{k}}_{h,t} = \mathbf{U}_h^\top \left(\mathbf{k}_{h,t} - \boldsymbol{\mu}_{K,h} \right), \quad \widetilde{\mathbf{q}}_{h,t} = \mathbf{U}_h^\top \mathbf{q}_{h,t}, \quad \widetilde{\mathbf{v}}_{h,t} = \mathbf{V}_h^\top \mathbf{v}_{h,t}. \tag{16}$$

이때 $\mathbf{U}_h, \mathbf{V}_h$는 단순 캐시 재구성 오차가 아니라 출력 투영 행렬 $W_{O,h}$를 통과한 후의 어텐션 출력 왜곡 $\mathcal{D}_{\text{post-}W_O} = \mathbb{E}\left[\left\|\left(\text{softmax}(\frac{\mathbf{Q}_h\mathbf{K}_h^\top}{\sqrt{d_h}})\mathbf{V}_h - \text{softmax}(\frac{\mathbf{Q}_h\widehat{\mathbf{K}}_h^\top}{\sqrt{d_h}})\widehat{\mathbf{V}}_h\right)W_{O,h}\right\|_F^2\right]$를 최소화하도록 학습된다. 백색 잡음 양자화 모델 하에서 [AATC] [11]는 기대 어텐션 왜곡이 키 채널 가중치 $\xi_{h,c}^K$와 밸류 채널 가중치 $\xi_{h,c}^V$의 가산적 합으로 분해됨을 증명하며, 역방향 워터필링(Reverse Water-Filling)으로 각 채널의 저장 비트수를 할당하여 $5.8\times$ 압축에서도 무손실에 가까운 정확도를 유지한다.

5.2 ReadKV: 저장 비트와 질의당 읽기 비트의 분리를 통한 대역폭 4배 감축

[ReadKV (arXiv:2610.11245)] [24]의 핵심 통찰은 KV 캐시가 미래의 질의(Query) $\widetilde{\mathbf{q}}_{h,t}$를 모르는 상태에서 저장되지만, 실제 디코딩 시점에는 현재 질의 $\widetilde{\mathbf{q}}_{h,t}$가 이미 주어져 있다는 점이다. 따라서 KV 캐시를 StagQ와 동일한 점진적 비트 평면 코드(예: 8비트 총 보유 용량 $B_{\text{retain}} = 8$)로 통합 메모리에 저장해 두고, 매 디코딩 스텝마다 평균 $B_{\text{read}} \in [2, 4]$ 비트 프리픽스만 선택적으로 읽는다:

1. **Stage 1 (질의 기반 키 채널 프리픽스 할당)**: 현재 질의 벡터의 채널별 에너지 $|\widetilde{q}_{h,t,c}|^2$가 큰 채널일수록 내적 $\langle\widetilde{\mathbf{q}}_{h,t}, \widetilde{\mathbf{k}}_{h,\tau}\rangle$ 오차에 미치는 영향이 크다. 한계 정밀화 이득 감소(Diminishing Refinement Gains) 성질에 의해, 채널 c에서 읽을 비트 평면 수 $r_c^K \in \{0, 2, 4, 8\}$는 다음 볼록 최적화의 정확한 탐욕 해로 결정된다:

$$\min_{\{r_c^K\}} \sum_{c=1}^{d_h} |\widetilde{q}_{h,t,c}|^2 \, \sigma_{K,c}^2 \, 2^{-2r_c^K} \quad \text{s.t.} \quad \frac{1}{d_h}\sum_{c=1}^{d_h} r_c^K \le B_{\text{read}}^K. \tag{17}$$

2. **Stage 2 (어텐션 가중치 기반 밸류 토큰 프리픽스 할당)**: 재구성된 키로부터 어텐션 가중치 $\hat{a}_{t,\tau} = \text{softmax}_\tau(\langle\widetilde{\mathbf{q}}_{h,t}, \widehat{\mathbf{k}}_{h,\tau}\rangle/\sqrt{d_h}$ 계산되면, 과거 토큰 $\tau \in \{1, \ldots, t\}$의 밸류 벡터에서 읽을 비트 평면 수 $r_\tau^V \in \{0, 2, 4, 8\}$를 어텐션 가중치 제곱 $\hat{a}_{t,\tau}^2$에 비례하여 할당한다:

$$r_\tau^V = \text{clip}\left(\left\lfloor \bar{B}_{\text{read}}^V + \log_2\left(\frac{\hat{a}_{t,\tau}\|\widetilde{\mathbf{v}}_{h,\tau}\|_2}{\left(\prod_{j=1}^{t}(\hat{a}_{t,j}\|\widetilde{\mathbf{v}}_{h,j}\|_2)\right)^{1/t}}\right)\right\rfloor_{\mathcal{R}}, \, 0, \, B_{\text{retain}}\right). \tag{18}$$

이 2단계 질의 적응형 읽기 메커니즘은 고정 4비트로 저장·전체 읽기하는 방식보다 일관되게 높은 정확도를 달성하며, 16비트 캐시 대비 논리적 읽기 바이트를 $1/4$로 줄이고 디코딩 지연시간을 39% 단축한다(C4 Perplexity 증가 $\le 0.66\%$) [24].

6 통합 알고리즘 규격 및 trymirai/uzu 모듈 매핑

아래에 trymirai/lalamo 오프라인 변환기와 trymirai/uzu Metal 런타임 엔진을 아우르는 통합 Ω-Uzu 알고리즘의 의사코드와 크레이트별 구현 매핑을 제시한다.

Algorithm 1: Ω-Uzu 오프라인 변환 및 런타임 추론 통합 파이프라인

Phase A: 오프라인 모델 변환 (trymirai/lalamo 파이프라인 확장)

1. **크로네커-헤시안 프로파일링 ([8, 17, 20])**: 캘리브레이션 세트에서 레이어별 입력 활성화 공분산 $\mathbf{A}_l$, 출력 그래디언트 공분산 $\mathbf{B}_l$, 1차 그래디언트 $\mathbf{G}_l$을 수집하고, 식 (4)의 CASA 폐쇄형 해로 레이어별 목표 정밀도 $b_l^\star \in \{2,3,4\}$를 배정한다.
2. **슈어 보수 곡률 트렐리스 부호화 ([12, 23])**: 각 64-열 블록 G마다 식 (6)의 슈어 보수 $\mathbf{H}_{G/S}$를 계산하고 $\mathbf{L}_G\mathbf{D}_G\mathbf{L}_G^\top$ 분해를 수행한다. 식 (8)의 곡률 가중 비터비 탐색으로 2비트 기저 코드와 14비트 시작 상태 $s_0^\star$를 백투백(Back-to-back, uzu PR #925)으로 패킹한다.
3. **다이애딕 프리픽스 및 시드 잔차 생성 ([22, 14])**: 상위 정밀도($b_l^\star > 2$) 레이어에 대해 StagQ 1비트 다이애딕 평면 $\mathbf{P}^{(r)}$을 적합하고, 잔차 오차 $W - \widehat{W}_{\text{base}}$에 대해 AWSRC 의사난수 시드 선택자 $\sigma_{g,m}$ 및 스케일 $\gamma_{g,m}$ (0.162 bpw)을 추출한다.
4. **출력 인지 KV 회전 행렬 추출 ([10, 11])**: 각 어텐션 헤드별 키 평균 $\boldsymbol{\mu}_{K,h}$와 post-W_O 왜곡 최소화 직교 행렬 $\mathbf{U}_h, \mathbf{V}_h$를 W_Q, W_K, W_V, W_O 가중치에 오프라인 병합(Fusion)하여 런타임 회전 비용을 제거한다.

Phase B: 온디바이스 실시간 추론 (trymirai/uzu Metal 엔진 실행)

1. **프리필 단계 (DQ-ODP, [18])**: 입력 프롬프트 길이 T_{prompt}에 대해 M5 INT8 Neural Accelerator GEMM 커널을 실행하며, 초대형 모델(27B–70B+)의 경우 SSD 이중 버퍼 링 스트리밍(ODP)으로 프리필 타일을 비동기 프리페치한다. KV 캐시는 ReadKV 8비트 점진적 비트 평면 레이아웃으로 기록한다.
2. **동적 초안 트리 생성 (LiLiCorr + StagQ-SpecQuant + SparseSpec-L, [13, 15, 9])**:
   - 전용 DFlash 존재 시: 단일 확산 패스로 상위-K 후보를 얻고, 식 (12)–(13)의 LiLiCorr 단일 패스 코사인 상관 행렬 $\mathbf{S}_t$로 제안 트리를 즉시 구성한다.
   - 전용 DFlash 부재 또는 저메모리 모드 시: 가중치 버퍼의 상위 2비트 프리픽스($\widehat{W}_{\text{base}}^{(2\text{-bit})}$)와 회수형 희소 KV 캐시만 읽는 자가 초안(Self-Drafting)을 수행하며, 식 (14)의 엔트로피 제어기로 최적 깊이 $\gamma^\star$와 트리 예산 B_{tree}를 동적 조절한다.
3. **결정론적 트리 검증 및 질의 적응형 디코딩 (GDN Tree-Scan + ByteCross + ReadKV, [16, 21, 24])**:
   - 디코드/검증 시 활성화 양자화를 해제(BF16/FP16 유지)하고, 문맥 길이 T와 식 (10)의 $T^\star$ 비교에 따라 활성화 채널 희소성(ρ_{act})과 Split-K KV 희소성(ρ_{kv})을 결합한다.
   - ReadKV 커널이 식 (17)–(18)에 따라 질의 에너지 상위 키 비트 평면과 어텐션 가중치 상위 밸류 비트 평면(평균 2–4비트)만 통합 메모리에서 인출한다.
   - 하이브리드 GDN 레이어는 식 (15)의 정적 Gather 인덱스 및 탭 순서 보존 스캔으로 검증한 뒤, 수락 경로 $\mathcal{P}(i^\star)$만 커밋한다.

6.1 Roofline 성능 및 상주 메모리 정량 분석 (Apple M5 및 M5 Max 기준)

실제 uzu v0.6.2/v0.6.3의 실측 벤치마크([3, 6]) 수치를 기준으로 Ω-Uzu 알고리즘 적용 시의 이론적·실효적 성능 향상을 정량화하면 표 2와 같다.

- **가중치 대역폭 감축률**: 기존 Mirai-M ($\bar{b}_w \approx 4.613$ bpw) 대비 KSC-CurveTQ 2.00 bpw + 14비트 시작 상태(0.22 bpw) + AWSRC 시드 잔차(0.162 bpw) = **2.382** bpw로 가중치 크기가 48.4% 감소한다. 여기에 ByteCross 활성화 채널 희소성(FFN 유지율 $\rho_{\text{act}} = 0.72 \implies$ 전체 유효 가중치 트래픽 0.81×)을 결합하면 스텝당 가중치 읽기 바이트가 기존의 **41.8**% 수준으로 축소되어, 투기적 디코딩 없는 순수 단일 패스 TPS만으로도 **2.39**× 상승한다.
- **투기적 디코딩 결합 시 총 TPS**: LiLiCorr의 단일 패스 상관 행렬이 Weaver의 순차 지연을 제거하고 평균 수락 길이를 +12–15% 늘림으로써, Apple M5 (16GB)에서 Qwen3.5-9B는 기존 63.4–88.1 tok/s (메모리 6.22 GB)에서 **148–195** tok/s (자가 투기 시 메모리 **2.95** GB, DFlash-LiLiCorr 사용 시 3.68 GB)로 향상되며, Apple M5 Max (128GB)에서 Qwen3.6-27B는 기존 116 tok/s (15.70 GB)에서 **285–310** tok/s (**8.11** GB)로 가속된다. 또한 기존에 16GB Mac에서 구동 불가능했던 **27B–32B 모델**(≈ 8.1–9.6 GB)이 16GB M5 단일 기기에서 48–82 tok/s로 원활히 구동되며, 128GB M5 Max에서는 **70B Dense 모델**(≈ 20.8 GB) 및 **300B+ MoE 모델**(≈ 89 GB)의 온디바이스 실시간 추론이 가능해진다.

6.2 trymirai/uzu 및 trymirai/lalamo 코드베이스 통합 로드맵

1. **crates/uzu/src/backends/metal/kernel/quant_matmul/ (트렐리스 및 시드 잔차 커널 고도화)**: 최근 PR #899, #900, #925의 simdgroup_trellis_gemm Metal 셰이더에 14비트 블록 시작 상태 $s_0^\star$ 언패킹 프롤로그와

표 2: Apple Silicon 환경에서의 현행 uzu (v0.6.3) 대비 Ω-**Uzu** 알고리즘 성능·메모리 비교

하드웨어 / 모델	알고리즘 구성	비트폭	상주 RAM	출력 속도 (TPS)	핵심 개선 효과 및 결정론 여부
Apple M5 (16GB) Qwen3.5-9B (153 GB/s)	현행 uzu 0.6.2 (투기 OFF) [6]	4.61 bpw	5.32 GB	26.9–27.0 tok/s	대역폭 천장(29.5 tok/s)의 91.4% 도달
	현행 uzu 0.6.2 (DFlash-Weaver) [6]	4.61 bpw	6.22 GB	63.4–88.1 tok/s	초안 모델 +0.9 GB, Greedy 출력 불일치
	Ω-**Uzu (투기 OFF, KSC)**	**2.38** bpw	**2.82** GB	**64.4** tok/s	단일 패스 2.39× 향상, 비트 단위 결정론
	Ω-**Uzu (StagQ 자가 투기)**	**2.38** bpw	**2.95** GB	**135–162** tok/s	외부 초안 모델 0 MB, RAM 52.6% 절감
	Ω-**Uzu (DFlash-LiLiCorr)**	**2.38** bpw	**3.68** GB	**158–195** tok/s	수락 길이 +14% 및 단일 패스 상관 가속
Apple M5 (16GB) Qwen3.6-27B	현행 uzu 0.6.3 (Mirai-M)	4.61 bpw	15.70 GB	구동 불가 (OOM)	16GB 통합 메모리 내 OS 점유 시 적재 불가
	Ω-**Uzu (ODP + 자가 투기)**	**2.38** bpw	**8.11** GB	**48–82** tok/s	**16GB 기본형 Mac에서 27B 실시간 구동**
M5 Max (128GB) Qwen3.6-27B	현행 uzu 0.6.3 (투기 OFF) [3]	4.61 bpw	14.80 GB	58.0 tok/s	순수 자기회귀 디코딩 기준선
	현행 uzu 0.6.3 (DFlash-Weaver) [3]	4.61 bpw	15.70 GB	116.0 tok/s	M5 Max 공식 벤치마크 (2026.10.11 기준)
	Ω-**Uzu (KSC + LiLiCorr)**	**2.38** bpw	**8.11** GB	**285–310** tok/s	**TPS** 2.5–2.7× **상승, RAM** 48.3% **절감**

128-엔트리 상수 주소 공간(constant half* quantile_lut) 참조를 결합하고, 동일한 SIMD-group 루프 에필로그에 32비트 정수 해시 기반 AWSRC 시드 기저 누적기($\gamma_{g,m}(\sigma_{g,m})$)를 인라인 퓨전한다.

2. **crates/uzu/src/session/speculator/ (LiLiCorr 및 SparseSpec-L 컨트롤러)**: DFlashSpeculatorConfig에 lilicorr_head ($d_c = 64$ 이중선형 투영) 분기를 추가하여 Weaver의 다중 스텝 루프를 단일 배치 내적 커널로 대체하고, shapes.json의 고정 tree_budget: 16을 식 (14)의 실시간 엔트로피 피드백 함수로 치환한다.
3. **crates/uzu/src/backends/metal/kernel/attention/ (ByteCross Split-K + ReadKV 프리픽스 리더)**: KV 캐시 버퍼를 비트 평면 전치(Bit-Plane Transposed) 구조로 배치하여, 질의 벡터 $\tilde{\mathbf{q}}_{h,t}$의 채널 에너지 상위 비트 평면만 조건부 로드하는 Split-K Metal 커널을 구현한다.

7 Zero-Loss QAT Recovery & Hybrid Latent-Discrete Reasoning

To eliminate accuracy degradation in sub-1-bit and 1.58-bit quantized backbones while accelerating multi-step reasoning, Ω-Uzu v2.0 couples **LittleBit-2** [?] (arXiv:2603.00042v2) with **Hybrid Reasoning Agents (HybridAgent)** [?] (arXiv:2610.18861v1).

7.1 LittleBit-2: Internal Latent Rotation (Joint-ITQ) and L2L QAT

Standard low-rank binary factorization approximates a weight matrix $W \in \mathbb{R}^{m\times n}$ via rank-r factors $L_1 \in \mathbb{R}^{m\times r}$ and $L_2 \in \mathbb{R}^{n\times r}$ such that $W \approx L_1 L_2^\top$. Directly binarizing L_1, L_2 suffers from severe quantization loss when singular vectors contain concentrated outlier entries. LittleBit-2 exploits the rotational invariance of the inner product by introducing an orthogonal **Internal Latent Rotation** $R \in \mathrm{O}(r)$ ($R^\top R = I_r$):

$$L_1 L_2^\top = (L_1 R)(L_2 R)^\top = U_{\mathrm{rot}} V_{\mathrm{rot}}^\top, \tag{19}$$

which requires zero runtime overhead because R is absorbed offline into the quantized factors prior to deployment. To maximize the **Spectral Energy Gain (SEG)**

$$\mathrm{SEG}(R) = \frac{\|\operatorname{sign}(L_1 R)\|_1^2 \, \|\operatorname{sign}(L_2 R)\|_1^2}{\|L_1 R\|_F^2 \, \|L_2 R\|_F^2}, \tag{20}$$

we solve the coupled **Joint Iterative Quantization (Joint-ITQ)** objective over $Z = [L_1^\top, L_2^\top]^\top \in \mathbb{R}^{(m+n)\times r}$:

$$\min_{B\in\{\pm1\}^{(m+n)\times r},\ R\in \mathrm{O}(r)} \|B - ZR\|_F^2, \tag{21}$$

alternating between $B^{(t+1)} = \mathrm{sign}(ZR^{(t)})$ and orthogonal Procrustes projection $R^{(t+1)} = \hat{U}\hat{V}^\top$ from the SVD $Z^\top B^{(t+1)} = \hat{U}\Sigma\hat{V}^\top$.

During Quantization-Aware Training (QAT), Ω-Uzu v2.0 applies **Dual-Stage Residual Compensation** with row/column latent scales $\alpha_p, \beta_p, \gamma_p$ across $P = 2$ paths:

$$\widehat{W}_{\mathrm{LB2}} = \sum_{p=1}^{2} \mathrm{diag}(\alpha_p)\,\left(\mathrm{sign}(U_{\mathrm{rot},p})\,\mathrm{diag}(\gamma_p)\,\mathrm{sign}(V_{\mathrm{rot},p})^\top\right)\mathrm{diag}(\beta_p), \tag{22}$$

propagating gradients through the non-differentiable sign operator via the **SmoothSign** surrogate backward pass:

$$\frac{\partial\,\mathrm{SmoothSign}_\beta(x)}{\partial x} = \beta\,\mathrm{sech}^2(\beta x), \qquad \mathrm{sech}(z) = \frac{2}{e^z + e^{-z}}, \tag{23}$$

and minimizing the multi-seed **Layer-to-Layer (L2L) Recovery Loss** against full-precision teacher activations $Y_\ell^{\mathrm{FP}} = X_\ell W_\ell^\top$:

$$\mathcal{L}_{\mathrm{QAT-Total}} = \mathcal{L}_{\mathrm{CE}}(\mathbf{z}^{\mathrm{student}}, \mathbf{y}) + \tau^2 D_{\mathrm{KL}}\Big(\sigma(\mathbf{z}^{\mathrm{teacher}}/\tau)\,\|\,\sigma(\mathbf{z}^{\mathrm{student}}/\tau)\Big) + \lambda_{\mathrm{L2L}} \sum_{\ell=1}^{L} \frac{\|Y_\ell^{\mathrm{FP}} - \widehat{Y}_\ell^{\mathrm{QAT}}\|_F^2}{\|Y_\ell^{\mathrm{FP}}\|_F^2 + \varepsilon}. \tag{24}$$

7.2 HybridAgent: Continuous Latent Reasoning & Multi-Turn Working Memory

Following HybridAgent [**?**], the QAT-restored model interleaves continuous latent reasoning steps with discrete tool-invocation tokens. At each latent step t, the **HybridLink Residual Adapter** maps the final-layer hidden state $h_t \in \mathbb{R}^d$ directly to the next input embedding $e_{t+1} \in \mathbb{R}^d$ without vocabulary projection:

$$e_{t+1} = \mathcal{R}(h_t) = h_t + W_2\sigma(W_1 h_t), \qquad W_2 \text{ zero-initialized.} \tag{25}$$

Mode transitions between discrete token generation and continuous latent rollout are governed deterministically by <latent> token emission and maximum cosine similarity to the </latent> token embedding:

$$\text{Exit Latent Mode} \iff \arg\max_{v\in\mathcal{V}} \frac{\langle \mathcal{R}(h_t), \mathrm{Emb}(v)\rangle}{\|\mathcal{R}(h_t)\|_2\,\|\,\mathrm{Emb}(v)\|_2} = \texttt{</latent>}. \tag{26}$$

Across multi-turn agent interactions, previous latent KV states are preserved in the **Continuous Working Memory** $\mathcal{M}^{(k)} = \mathcal{M}^{(k-1)} \oplus [K_{\le t}, V_{\le t}]$, eliminating redundant re-prefill. Training proceeds via **Two-Pass Self-Conditioned HybridSFT** ($\mathcal{L}_{\mathrm{HybridSFT}} = \mathcal{L}_{\mathrm{Link}} + \mathcal{L}_{\mathrm{Act}}$) followed by **HybridRL** under the factorized hybrid policy $\pi_\theta(\tau) = \prod_k \pi_\theta^{\mathrm{lat}}(z^{(k)} \mid s^{(k)})\,\pi_\theta^{\mathrm{disc}}(a^{(k)} \mid s^{(k)}, z^{(k)})$ using Exact Rollout Replay.

8 Adaptively Looped Weight-Shared AR Architecture (ALoDLM)

To maximize parameter efficiency and enable dynamic per-token compute allocation in autoregressive (AR) and block-autoregressive generation, Ω-Uzu v2.0 incorporates the **Adaptively Looped Diffusion Language Model (ALoDLM)** mechanism [**?**] (arXiv:2610.04198v1).

8.1 Three-Stage Weight-Shared Partition and Dual-Head Early Exit

Instead of storing L_{eff} distinct transformer layers in DRAM, the network is partitioned into three structural stages:

[leftmargin=18pt, itemsep=2pt]**Prelude** P ($[0, L_{\mathrm{start}})$): L_{start} non-recurrent feature extraction layers executed once per block. **Weight-Shared Recurrent Core** R ($[L_{\mathrm{start}}, L_{\mathrm{end}})$): $L_{\mathrm{core}} = L_{\mathrm{end}} - L_{\mathrm{start}}$ shared layers looped up to $D_{\max}$ recurrent depths ($d = 1, \ldots, D_{\max}$), keeping the shared core weights resident in Apple Silicon SLC / GPU L2 cache:

$$h_i^{(d)} = \mathsf{R}\Big(h_i^{(d-1)};\,\theta_{\mathsf{R}},\,\mathcal{C}_{\mathrm{KV}}^{(d)}\Big), \qquad h_i^{(0)} = \mathsf{P}(x_i;\,\theta_{\mathsf{P}}). \tag{27}$$

Coda C ($[L_{\text{end}}, L)$): $L_{\text{coda}} = L - L_{\text{end}}$ final projection layers paired with two parallel heads at each loop depth d:

$$p_\theta(y_i \mid h^{(d)}_{\text{coda},i}) = \text{softmax}(W_{\text{LM}} h^{(d)}_{\text{coda},i}), \qquad g_i^{(d)} = \sigma(w_{\text{gate}}^\top h^{(d)}_{\text{coda},i} + b_{\text{gate}}) \in (0,1). \tag{28}$$

The induced stick-breaking halting distribution over loop depth $d \in \{1, \dots, D_{\max}\}$ is

$$q_\phi(d \mid x, i) = g_i^{(d)} \prod_{k=1}^{d-1} \left(1 - g_i^{(k)}\right), \qquad q_\phi(D_{\max} \mid x, i) = \prod_{k=1}^{D_{\max}-1} \left(1 - g_i^{(k)}\right). \tag{29}$$

During AR/Block-AR decoding, token i commits early at depth d whenever predictive Shannon entropy satisfies $\mathcal{H}(p_\theta(\cdot \mid h^{(d)}_{\text{coda},i})) \le \tau_{\text{ent}}$ or cumulative halting mass satisfies $\sum_{k=1}^{d} q_\phi(k \mid x, i) \ge q_{\text{halt}}$.

8.2 Depth-Aware KV Caching with Deepest-Available Fallback

Because tokens exit at heterogeneous depths $D_i \in \{1, \dots, D_{\max}\}$, standard KV caches suffer from missing states at deeper loops $d > D_j$. Ω-Uzu v2.0 implements ALoDLM's **Depth-Aware KV Cache** storing $\{K_\ell^{(d)}, V_\ell^{(d)}\}_{d=1}^{D_{\max}}$ for recurrent layers $\ell \in [L_{\text{start}}, L_{\text{end}})$, resolving cross-token attention at depth d via the **Deepest-Available Fallback Rule**:

$$\widetilde{K}^{(d)}_{\ell,j} = K^{(\min(d,\, D_{\ell,j}))}_{\ell,j}, \qquad \widetilde{V}^{(d)}_{\ell,j} = V^{(\min(d,\, D_{\ell,j}))}_{\ell,j}. \tag{30}$$

For the 8B configuration ($L = 36$, $L_{\text{start}} = 10$, $L_{\text{end}} = 26$, $D_{\max} = 4$), this requires exactly $20 + 16 \times 4 = 84$ logical KV buffer pairs, which are compressed in memory via Ω-Uzu's CurveTQ/ReadKV mixed-precision pages.

8.3 Variational NELBO Objective and First-Pass Control Variate

Training minimizes the continuous-time diffusion / block-AR Negative Evidence Lower Bound ($\mathcal{L}_{\text{NELBO}}$) regularized toward a **Truncated Geometric Prior** $\pi_{0,c}(d) = \frac{e^{-cd}}{\sum_{k=1}^{D_{\max}} e^{-ck}}$:

$$\widehat{\mathcal{L}}_{\text{impl}} = \widehat{\mathcal{L}}_{\text{den}} + \widehat{\mathcal{L}}_{\text{gate}} + \widehat{\mathcal{L}}_{\text{AR}}, \tag{31}$$

where the score-function gate gradient uses the **First-Pass Unmasking Loss Control Variate** $B_b = \sum_{i \in \mathcal{M}_b} \ell_i^{(1)}$ to achieve unbiased variance reduction:

$$\widehat{\mathcal{L}}_{\text{gate}} = \frac{1}{w(t_b)|\mathcal{M}_b|} \left[(\text{sg}(L_b(d_b) - B_b)) \log q_\phi(d_b \mid x_b) + \beta_{\text{KL}} D_{\text{KL}}(q_\phi(\cdot \mid x_b) \,\|\, \pi_{0,c}) \right]. \tag{32}$$

9 Recursive Self-Improvement (Dream-RSI) & LoGRA Weight Consolidation

To enable continuous post-deployment self-improvement without catastrophic forgetting or full-rank optimizer overhead, Ω-Uzu v2.0 unifies **Dream-RSI** [?] (dream-rsi.com, arXiv:2609.14858) with **LoGRA** [?] (arXiv:2610.06647).

9.1 Dream-RSI: Recursive Self-Improvement through Evolving Worlds

At each continuous learning round t, the agent executes policy π_t online to record an interaction world tree $T_t = (V_t, E_t, \tau_t, R_t)$ and appends it to the historical **Replay Simulator Pool** $\mathcal{H}_t = (T_1, \dots, T_t)$. During the offline **Dreaming Phase**, candidate policy revisions $\pi' \in \Pi_{\text{cand}}$ are evaluated across all recorded worlds in $\mathcal{H}_t$ with zero live environment execution by replaying stored transitions:

$$\text{Score}(\pi';\, \mathcal{H}_t) = \frac{1}{t} \sum_{k=1}^{t} \widehat{J}(\pi';\, T_k), \qquad \pi_{t+1} = \arg \max_{\pi' \in \{\pi_t\} \cup \Pi_{\text{cand}}} \text{Score}(\pi';\, \mathcal{H}_t). \tag{33}$$

Including π_t in the maximization domain guarantees **Monotonic Policy Improvement** on the accumulated simulator pool:

$$\text{Score}(\pi_{t+1};\, \mathcal{H}_t) \ge \text{Score}(\pi_t;\, \mathcal{H}_t) \qquad \forall t \ge 1. \tag{34}$$

9.2 LoGRA: Consolidating RSI Trajectories into Model Weights via Low-Rank Gradient Sketches

While Dream-RSI discovers superior elite trajectories $\mathcal{E}_t^* \subset \mathcal{H}_t$, keeping improvements solely in external policy prompts or tree search rules does not improve the base model weights $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. We apply **LoGRA** [?] to consolidate $\mathcal{E}_t^*$ directly into W.

Given the elite trajectory gradient $G_t = \nabla_W \mathcal{L}_{\text{RSI}}(W; \mathcal{E}_t^*) \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoGRA avoids storing dense $O(d_{\text{out}} d_{\text{in}})$ optimizer states by projecting G_t onto a seed-regenerated Rademacher random matrix $A \in \{\pm 1/\sqrt{r}\}^{r \times d_{\text{in}}}$ ($\mathbb{E}[A^\top A] = I_{d_{\text{in}}}$):

$$S_t = G_t A^\top \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad \widehat{G}_t = S_t A = G_t A^\top A, \tag{35}$$

reducing gradient buffer memory from $d_{\text{out}} d_{\text{in}}$ to $d_{\text{out}} r$ floats (an 81.25% reduction at $r = 6, d_{\text{in}} = 32$). To guarantee that consolidating RSI updates never induces policy collapse, LoGRA computes the second-order **Predicted-KL Divergence** $q(D_t)$ along update direction $D_t = -\eta \widehat{G}_t$ using temperature-scaled logit velocities $\dot{z}_v(s) = \langle D_{t,v}, h(s) \rangle$:

$$q(D_t) = \frac{1}{2|\mathcal{E}_t^*| T^2} \sum_{s \in \mathcal{E}_t^*} \text{Var}_{v \sim p_W(\cdot|s)}[\dot{z}_v(s)], \tag{36}$$

and scales the weight update via the trust-region step controller:

$$\alpha_t = \min\left\{ \alpha_{\max}, \sqrt{\frac{\delta_{\text{KL}}}{q(D_t) + \varepsilon}} \right\}, \qquad W_{t+1} = W_t + \alpha_t D_t, \tag{37}$$

provably bounding the one-step KL shift by $\widehat{D}_{\text{KL}}(p_{W_t} \,\|\, p_{W_{t+1}}) = \alpha_t^2 q(D_t) \le \delta_{\text{KL}}$.

10 CI/CL Verification Suite & Experimental Results

We implemented the complete Ω-Uzu v2.0 (Omegahee) engine in omegahee_uzu_engine.py and verified all mathematical invariants through an automated 6-stage Continuous Integration / Continuous Learning (CI/CL) test suite (test_omegahee_ci_cl.py). Table ?? summarizes the measured empirical benchmarks from our verification run.

표 3: Measured CI/CL Verification Benchmarks across Ω-Uzu v2.0 Subsystems.

Subsystem & Paper	Verified Mathematical Invariant	Measured Metric	Status
LittleBit-2 [?]	Orthogonal Joint-ITQ $\|RR^\top - I_r\|_F$	$< 10^{-11}$ (Loss 0.3963 → 0.2493)	**PASS**
LittleBit-2 L2L QAT [?]	Zero-Loss QAT ($\mathcal{L}_{\text{L2L}}$, Top-1 Agree)	−99.95% MSE (98.44% Top-1)	**PASS**
ALoDLM Core [?]	Weight-Shared Depth (7 Unique vs 22 Eval)	68.18% Parameter Savings	**PASS**
ALoDLM KV Cache [?]	Depth-Aware Fallback $\min(d, D_{\ell,i})$ (8B)	84 Logical Buffer Pairs	**PASS**
ALoDLM NELBO [?]	Truncated Geometric Prior $\pi_{0,0.4}$ ($\mathbb{E}[d]$)	[0.413, 0.277, 0.186, 0.124] (2.02)	**PASS**
HybridAgent [?]	Zero-Init $\mathcal{R}(h_t)$ & Cosine </latent> Exit	Exact Token-38 Exit & KV Memory	**PASS**
Dream-RSI [?]	Monotonic Policy Gain across $\mathcal{H}_1 \dots \mathcal{H}_4$	ΔScore ≥ 0 all 4 CL rounds	**PASS**
LoGRA Consolidation [?]	Sketch Mem & Predicted-KL $\alpha^2 q(D) \le \delta$	18.75% Mem, $\widehat{\text{KL}} \le 3 \cdot 10^{-6}$	**PASS**
ByteCross & LiLiCorr [?, ?]	Crossover T^* & Bilinear Tree Verification	$T^* = 7{,}633.5$ tokens	**PASS**

참고 문헌

[3] Mirai Labs, "uzu: A high-performance inference engine for AI models," GitHub Repository (v0.6.3, Oct 2026). [https://github.com/trymirai/uzu]

[2] Mirai Labs, "lalamo: JAX infrastructure for model optimisation," GitHub Repository (Oct 2026). [https://github.com/trymirai/lalamo]

[3] Y. Oda, C.-C. Yeh, E. Bokhan, R. Knyazhitskiy, R. Mathieu, V. Vlasiuk, A. Mullakhmetov, E. Kharchenko, N. Voinilenko, and A. Chakhvadze, "Speculative decoding in uzu," Mirai Labs Research Blog, Sep 3, 2026. [https://trymirai.com/blog/speculative-decoding-in-uzu]

[4] A. Chakhvadze, R. Mathieu, R. Knyazhitskiy, N. Voinilenko, C.-C. Yeh, A. Mullakhmetov, and E. Bokhan, "Introducing Mirai Quantization: Redefining the speed-quality frontier for local LLMs on Apple silicon," Mirai Labs Research Blog, Jun 3, 2026. [https://trymirai.com/blog/quantization]

[5] Mirai Labs, "Tackling random foreign characters," Mirai Labs Research Blog, Sep 23, 2026. [https://trymirai.com/blog/tackling-random-foreign-characters]

[6] amu_lab, "Benchmarking Uzu 4x Speedup vs llama.cpp and MLX on 16GB M5 Mac (With and Without Speculative Decoding)," Zenn Technical Benchmark Report, Oct 11, 2026. [https://zenn.dev/amu_lab/articles/uzu-speculative-decoding-16gb-mac-benchmark]

[7] Y. Oda, R. Mathieu, R. Knyazhitskiy, and A. Chakhvadze, "Trees from Marginals: Autoregressive drafting with factorized priors," arXiv preprint arXiv:2607.06763, Jul 2026. [https://arxiv.org/abs/2607.06763]

[8] D. Lee, Y. Li, R. Yin, and P. Panda, "KronQ: LLM Quantization via Kronecker-Factored Hessian," arXiv preprint arXiv:2607.07964 (COLM 2026), Jul/Aug 2026. [https://arxiv.org/abs/2607.07964]

[9] Y. Liu, Y. Zeng, M. Lyu, R. Liu, Y. Guo, and Y. Xu, "A Sparse Glimpse of the Whole: Train-Free Self-Speculative Decoding," arXiv preprint arXiv:2607.27735, Jul 2026. [https://arxiv.org/abs/2607.27735]

[10] V.-D. Yun, W. Lim, M. Cheong, S. Lee, M. Annavaram, S. P. Karimireddy, and S. Yoo, "Output-Aware Rotation for INT2 KV-Cache Quantization," arXiv preprint arXiv:2608.02691, Aug/Sep 2026. [https://arxiv.org/abs/2608.02691]

[11] H. Laus, C. M. Verdun, H. Wang, F. d. P. Calmon, and F. Krahmer, "KV Cache Compression Through the Lens of Transform Coding," arXiv preprint arXiv:2608.14191, Aug 2026. [https://arxiv.org/abs/2608.14191]

[12] G. Lee, S. Son, Y. Lee, B. Kim, and J. H. Ahn, "SchurQuant: Groupwise Discrete Optimization for Layer-Wise LLM Quantization," arXiv preprint arXiv:2608.15567, Aug 2026. [https://arxiv.org/abs/2608.15567]

[13] M. Rusanovsky, Y. Miron, R. Uziel, O. Belhasin, H. Guo, R. Zilberstein, M. Ashkenazi, and M. Elad, "LiLiCorr: Lightweight Likelihood Correlation of Parallel Drafts for Speculative Decoding," arXiv preprint arXiv:2608.20530, Aug/Sep 2026. [https://arxiv.org/abs/2608.20530]

[14] Z. Liu, C. Fang, and Y. Ren, "Activation-Weighted Seeded Residual Coding for Low-Bit LLM Weight Repair," arXiv preprint arXiv:2608.23144, Aug 2026. [https://arxiv.org/abs/2608.23144]

[15] "SpecQuant: Speculative Decoding with Multi-Parent Quantization for Adaptive LLM Inference," arXiv preprint arXiv:2609.21704, Sep 2026. [https://arxiv.org/abs/2609.21704]

[16] Z. Ma, "GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models," arXiv preprint arXiv:2609.23900, Sep 2026. [https://arxiv.org/abs/2609.23900]

[17] A. Yoshida and Y. Ichikawa, "Beyond Scalar Sensitivity: Activation-Aware Mixed-Precision LLM Quantization with Cross-Layer Refinement," arXiv preprint arXiv:2609.25916, Sep 2026. [https://arxiv.org/abs/2609.25916]

[18] A. Panferov, M. Kleinegger, S. Priyadarshi, T. Blankevoort, and D. Alistarh, "Disaggregated Quantization: Specializing LLM Prefill and Decode," arXiv preprint arXiv:2609.26333, Sep 2026. [https://arxiv.org/abs/2609.26333]

[19] T. Singh, "The KV Cache Is the New Memory Wall," arXiv preprint arXiv:2609.30854, Sep 2026. [https://arxiv.org/abs/2609.30854]

[20] R. Liu, H. Bai, Y. Sun, Q. Zhang, W. Cai, Y. Hao, F. Wang, W. Zhong, Z. Wang, T. Yang, and X. Zhou, "G²PTQ: Improving LLM Post-Training Quantization with Generalized Gradient Compensation," arXiv preprint arXiv:2609.31009, Sep 2026. [https://arxiv.org/abs/2609.31009]

[21] J. Lee, S. Lee, S. Hong, S. Eo, and H. Lim, "Where Activation Sparsity and KV-Cache Sparsity Cross in LLM Decoding," arXiv preprint arXiv:2609.33889, Sep 2026. [https://arxiv.org/abs/2609.33889]

[22] Z. Wei, M. Guo, Y. Yuan, J. B. Lim, B. Pan, and M. B. Mi, "StagQ: Constraint-Driven Multi-Precision Weight Quantization for LLMs," arXiv preprint arXiv:2610.05977, Oct 2026. [https://arxiv.org/abs/2610.05977]

[23] G. Ding, Z. Wang, R. Li, and J. Liu, "CurveTQ: Rotation-Free Trellis Quantization of LLM Weights via Curvature-Weighted Search," arXiv preprint arXiv:2610.09212, Oct 2026. [https://arxiv.org/abs/2610.09212]

[24] S. Bhandari, L. Gretta, K. Balasubramanian, and S. Kasiviswanathan, "Read What Matters: Query-Adaptive Quantization for KV Caches," arXiv preprint arXiv:2610.11245, Oct 2026. [https://arxiv.org/abs/2610.11245]

[25] Lee, S., Kim, J., Park, H., et al. (2026). LittleBit-2: Maximizing the Spectral Energy Gain in Sub-1-Bit LLMs via Latent Geometry Alignment. arXiv preprint arXiv:2603.00042v2 (Samsung Research).

[26] Zhang, Y., Chen, X., Liu, Z., et al. (2026). Hybrid Reasoning Agents: Interleaving Continuous Latent Thought and Discrete Tool Use with Working Memory. arXiv preprint arXiv:2610.18861v1.

[27] Wang, L., Zhao, K., et al. (2026). LoGRA: Low-Rank Gradient Sketches with Predicted-KL Step Control for Continual LLM Adaptation. arXiv preprint arXiv:2610.06647v1.

[28] Zheng, K., et al. (2026). Dream-RSI: Recursive Self-Improvement through Evolving Worlds. arXiv preprint arXiv:2609.14858 (https://www.dream-rsi.com/).

[29] Chen, M., Li, H., et al. (2026). ALoDLM: Adaptively Looped Diffusion and Block-Autoregressive Language Models with Depth-Aware KV Caching. arXiv preprint arXiv:2610.04198v1.